

CS 4530: Fundamentals of Software Engineering

Module 7.1: Software Development Processes

Adeel Bhutta and Mitch Wand
Khoury College of Computer Sciences

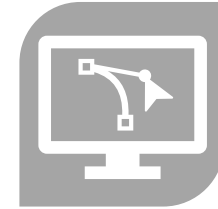
So, software engineering must encompass:



PEOPLE



PROCESSES

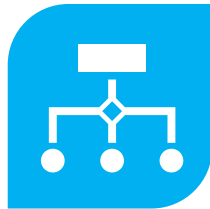


PROGRAMS

PLANNING



ORGANIZING



IMPLEMENTING



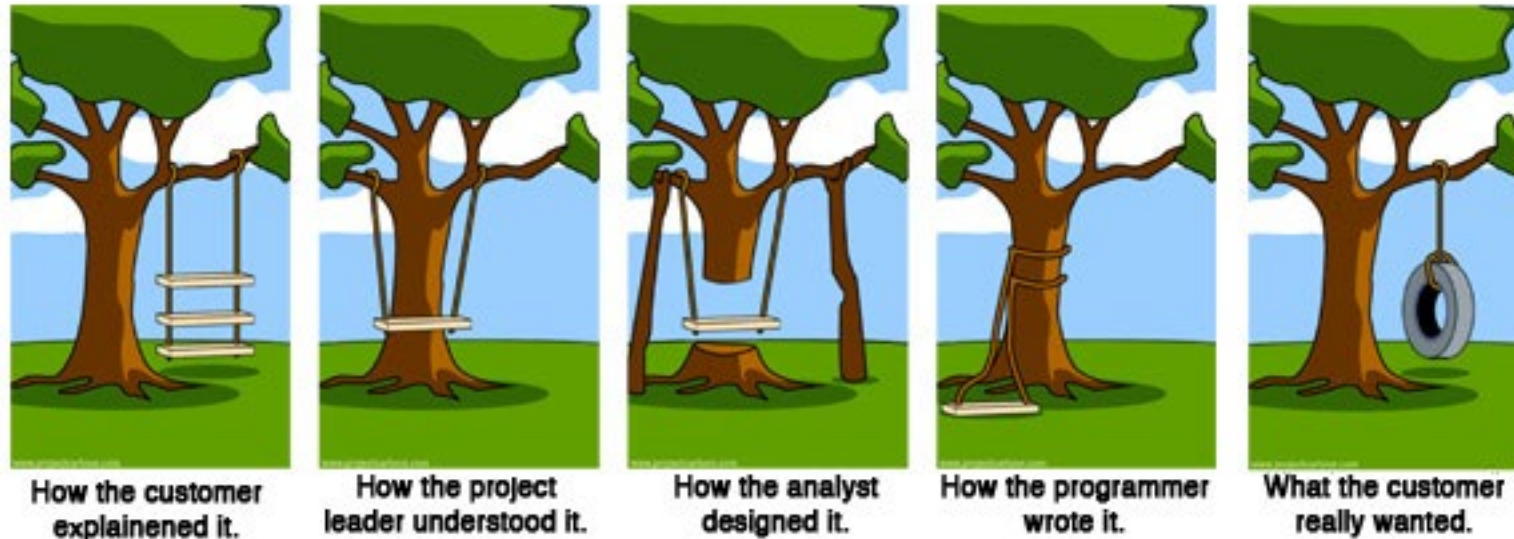
We are here today!

Learning Goals for this Lesson

- At the end of this lesson, you should be able to
 - Know the basic characteristics of the waterfall software process model
 - Be able to explain when the waterfall model is appropriate and when it is not
 - Understand how the waterfall and agile models manage risk
 - Be able to explain how agile process instill quality, including through test driven development

Review:

How to make sure we are building the right thing

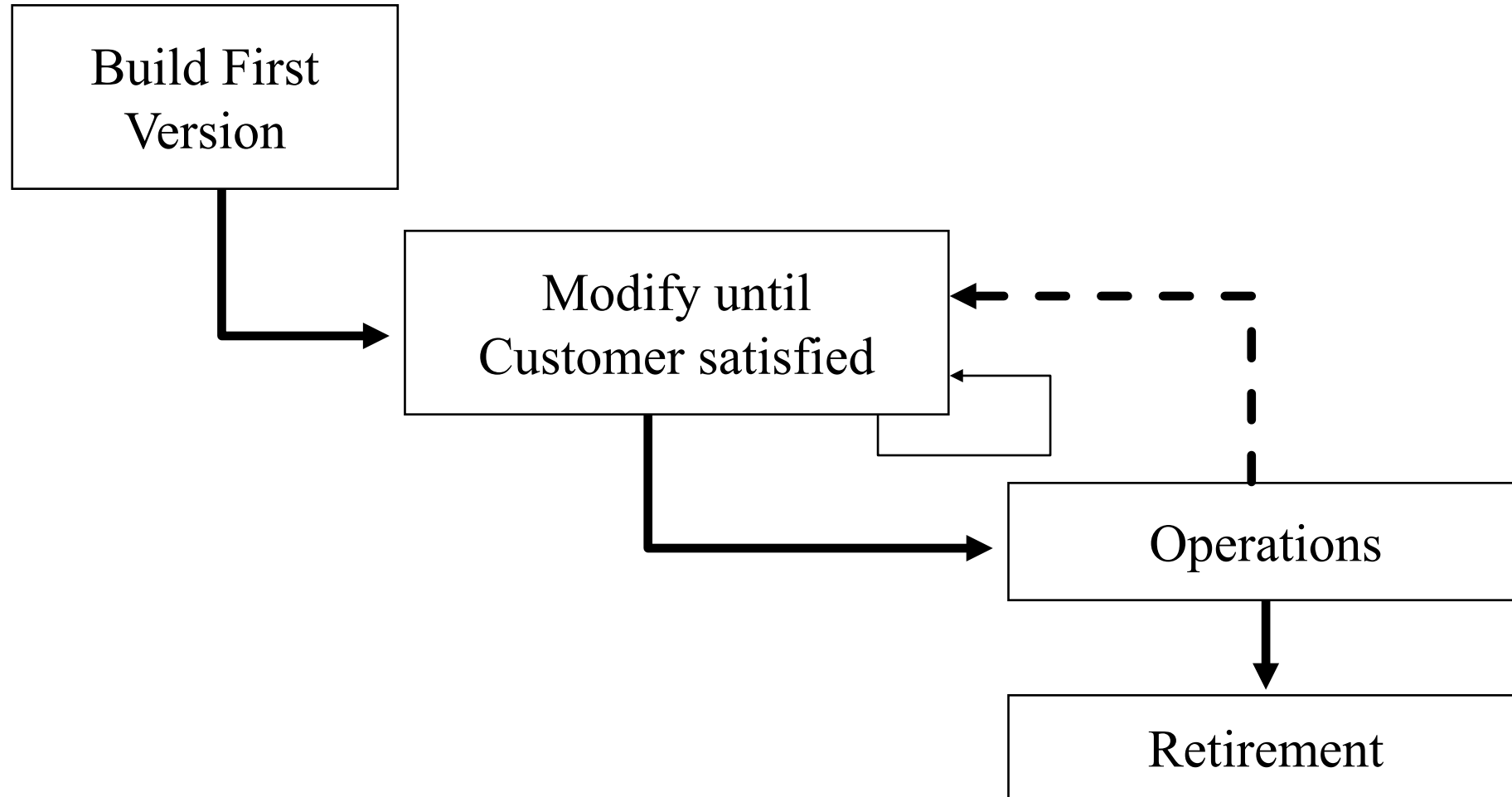


Requirements
Analysis

Planning &
Design

Implementation

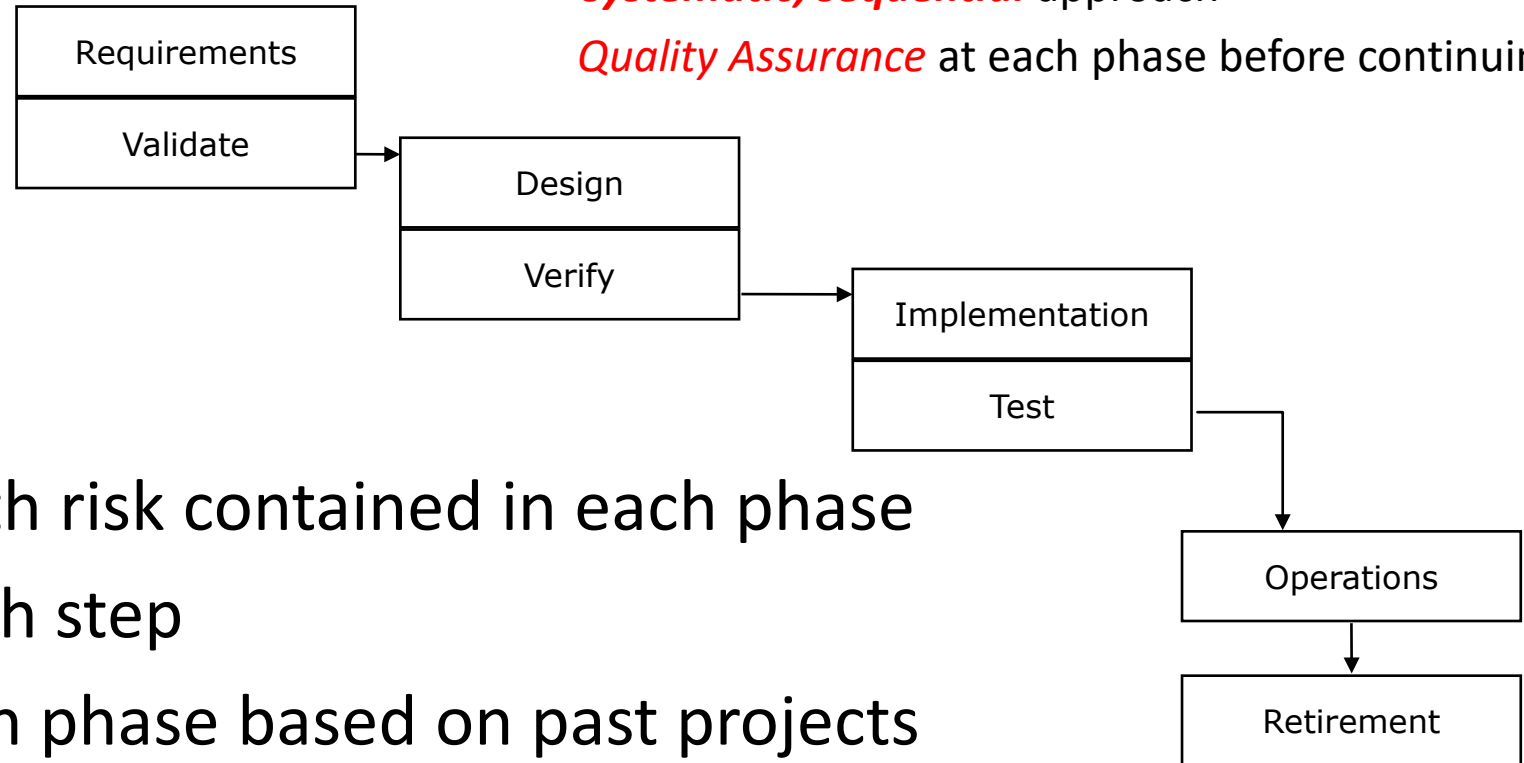
Software Process: Code + Fix



Waterfall Process Improves on Code + Fix (~1970s)

systematic, sequential approach

Quality Assurance at each phase before continuing



- Measurable progress with risk contained in each phase
- Quality Assurance at each step
- Possible to estimate each phase based on past projects
- Division of labor: Natural segmentation between phases

Waterfall Model adds process *overhead* and produces *Wasted Work Product*

Since formal quality assurance happens at each phase, it's necessary to produce extremely detailed documentation (requirements, design, testing)

But

Documentation produced per requirements was never read, Elaborate architectural designs never used



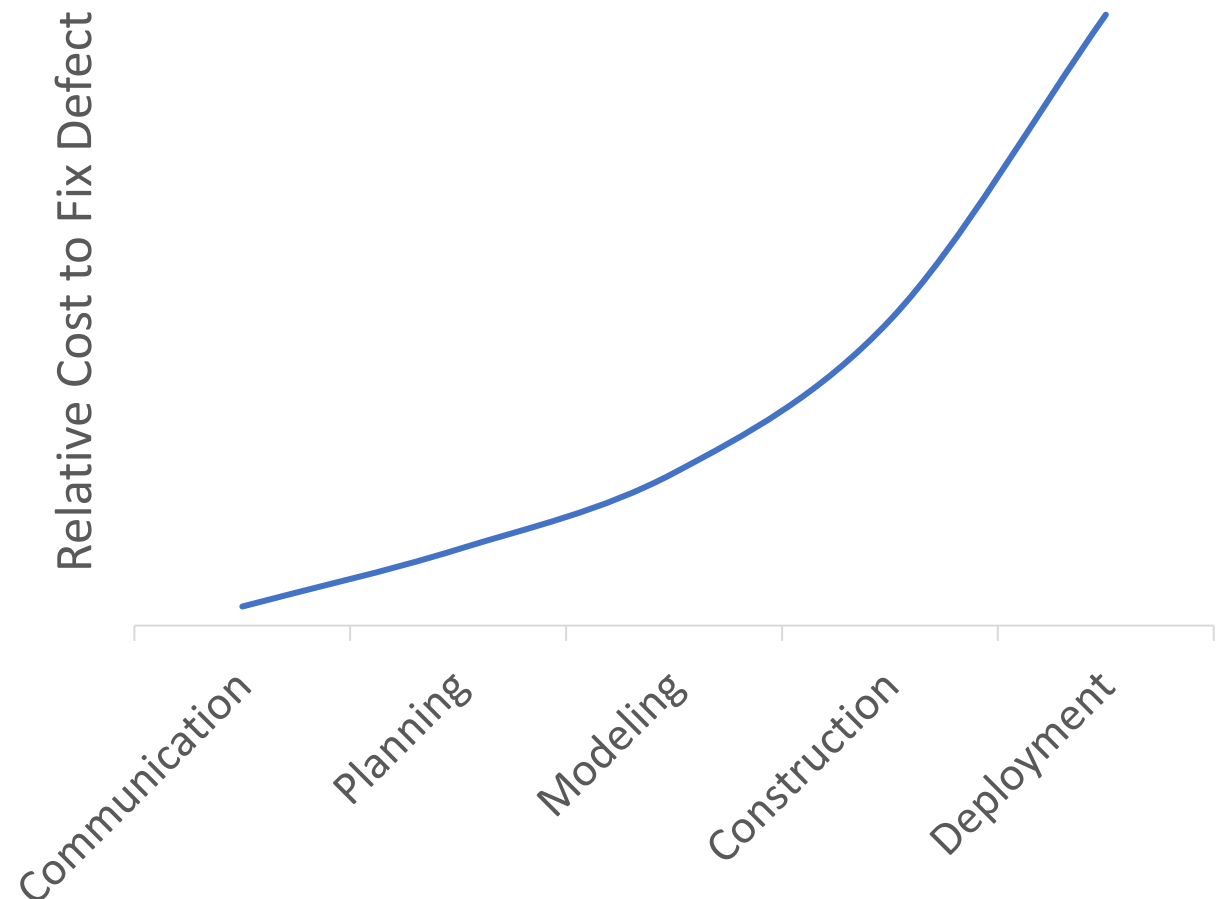
Waterfall Model: Applications

- What projects would this work well in?
 - Projects with tremendous uncertainty or risk
 - Projects with long time-to-market
 - Projects that need extensive QA of requirements and design
 - Projects for which the expense of the planning is worth it
 - Classic examples: military/defense
 - Warship that needs to have component interfaces last 80 years
 - Spacecraft?

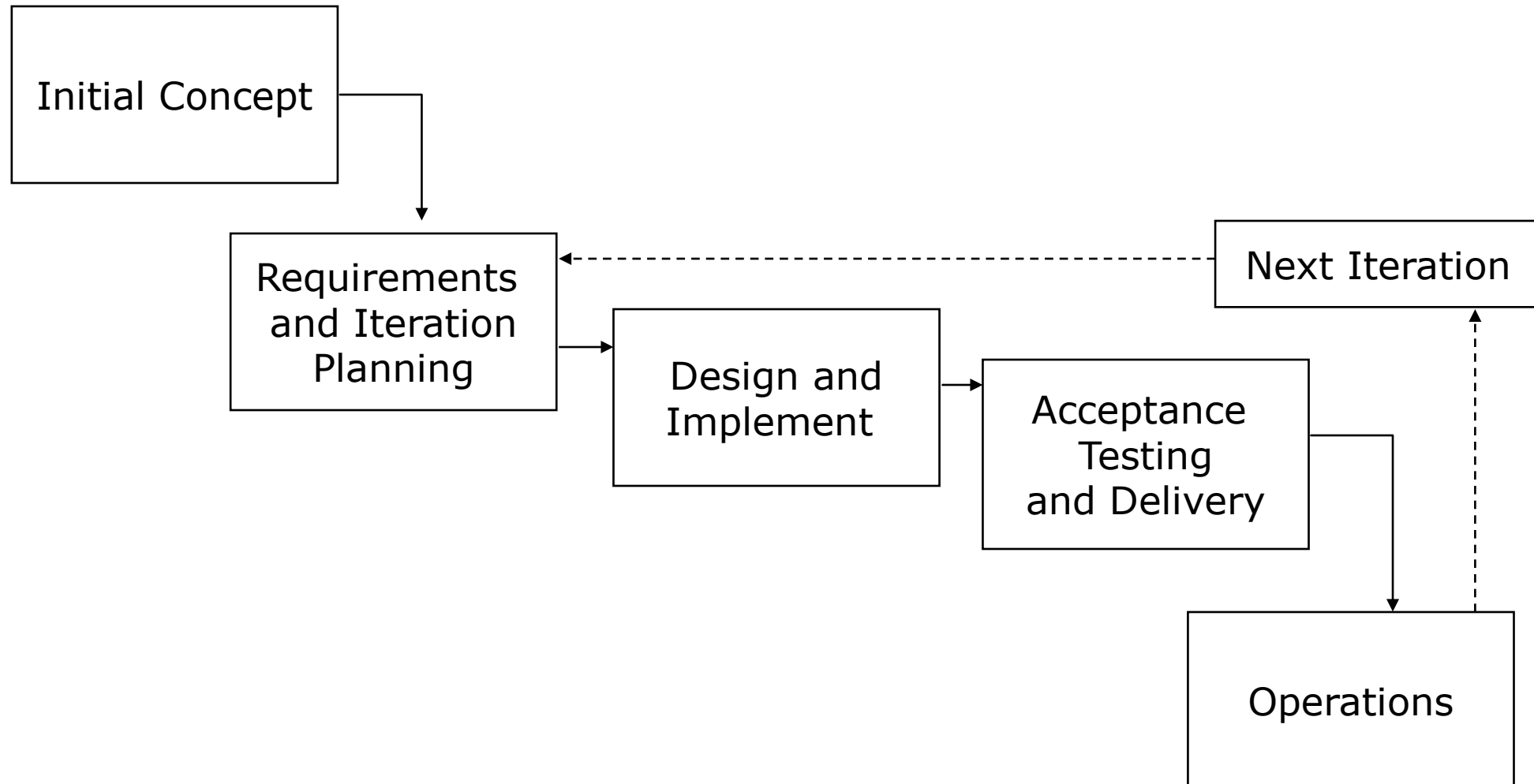
Waterfall Model: Risk Assumptions

The cost to fix a defect grows exponentially with each development phase

- Waterfall Model Reduces Risk by **Preventing Change**
 - No way to go “back up”



Iterative Process (~1980s) are Waterfall Variations



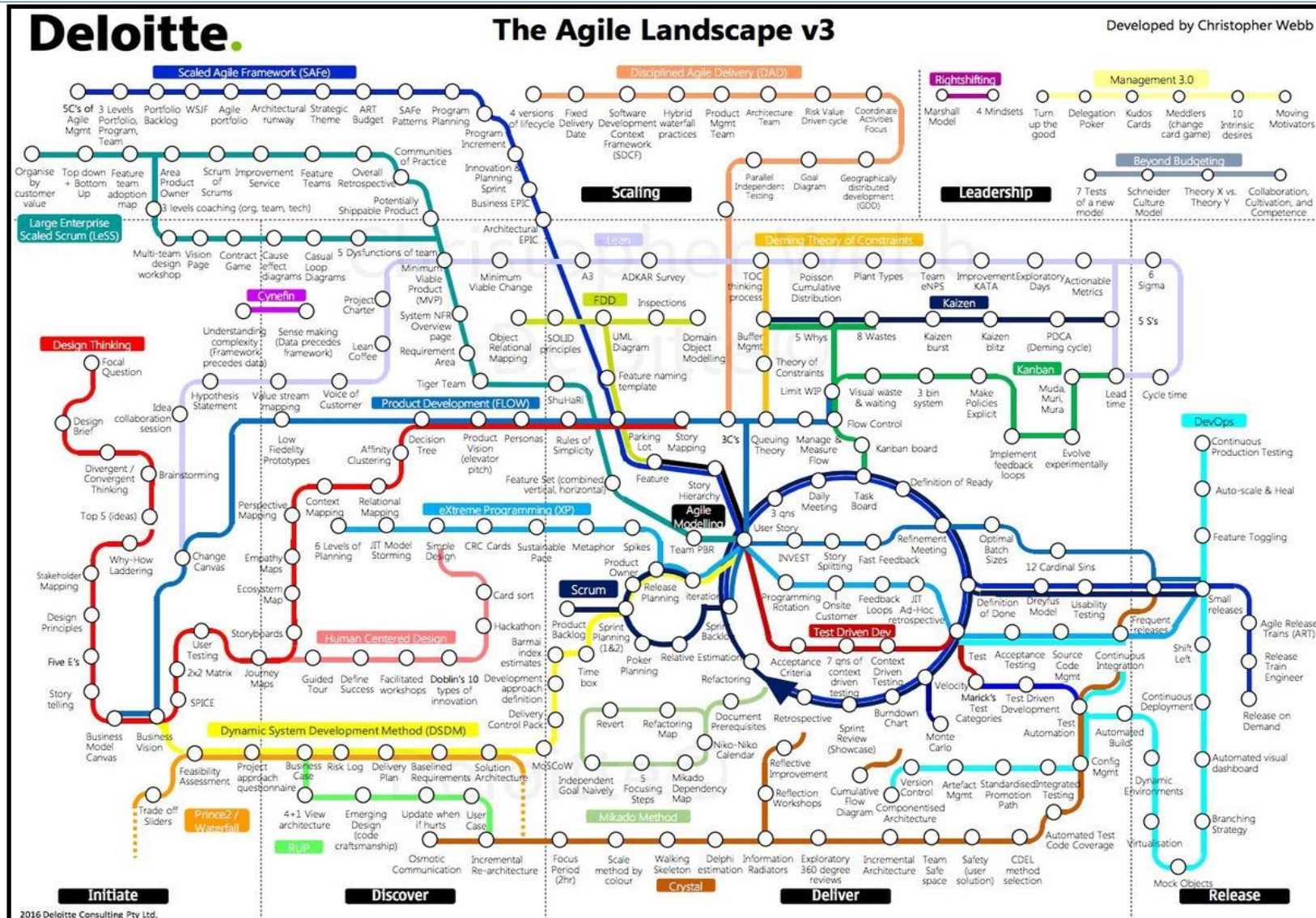
Agile Manifesto (~2000)

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions	over processes and tools
Working software	over comprehensive documentation
Customer collaboration	over contract negotiation
Responding to change	over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Warning: Agile can be a buzzword

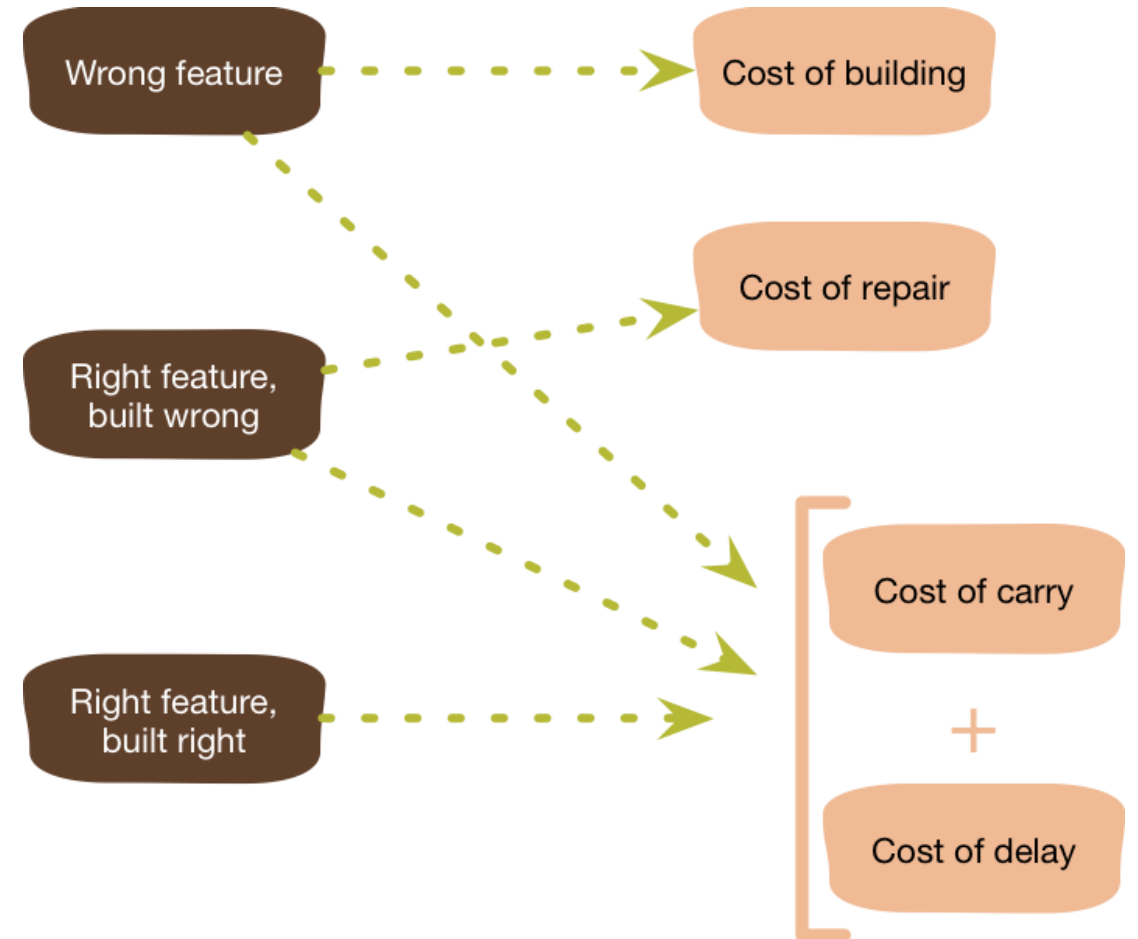


The Agile Model Reduces Risk by Embracing Change

- The Waterfall philosophy:
 - "The project is too large and complex, and it will take months (or years!) to plan, so once we come up with the plan, that plan can not change"
 - Reduce risk by proceeding in stages
- The Agile philosophy:
 - The project is too large and complex, it is unlikely that we will know exactly what we need right now, and to some extent, we are inventing something new. We think that as we make it, we will figure it out as we go"
 - Reduce risk by limiting time on any one stage; then reassess. ("time-boxing")

Agile Practice: YAGNI

- YAGNI – “You Aren’t Going To Need It”
- Do not prematurely plan or implement features
- Why? Uncertainty *in what we actually need*
- Focus on *prioritization*, independent of estimation



Graphic: Martin Fowler

Agile improves on other approaches

Compare to problems in waterfall:

- Requirements that become obsolete
 - Don't make detailed requirements until you need them
- Elaborate architectural designs never used
 - Don't design until you need
- Code that sits around not integrated and tested in production environment, eventually discarded
 - Integrate and test continuously
- Documentation produced per requirements, but never read
 - Don't require documentation

Or only as much documentation as you really need.

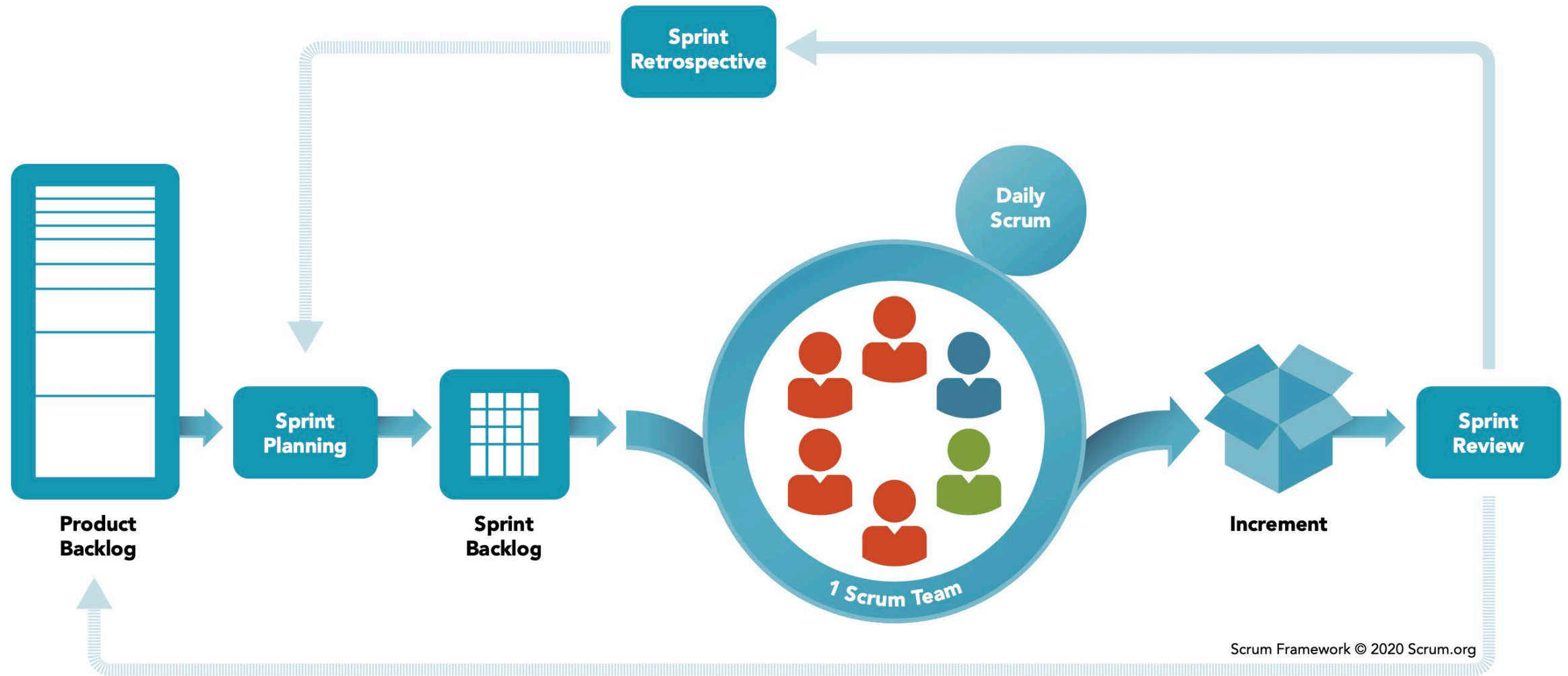
Agile Practice: Everyone is Responsible for Quality

- “Collective ownership”
- Requirements (user stories) are developed collaboratively with customer, and are *negotiable* (INVEST+E qualities)
- Functional and non-functional correctness is checked *on the cheap*, and often
- Developers improve code anywhere in the system if they see the opportunity
- Many parallels with “Toyota Process System;” where workers were ***empowered to improve processes***. A variety of other software processes developed in the 90’s share these basic values

Agile requires quality assurance processes

- Quality is everyone's responsibility
- Multiple processes work together to ensure quality:
 - unit testing/TDD
 - mix of unit tests & integration tests (we'll see more of this)
 - code review
 - continuous integration
 - continuous deployment (A/B, canaries, etc.)
 - quality includes non-functional requirements (resource consumption, response time) or generally speaking usability, extensibility, maintainability, etc.

Scrum is the most common approach to organizing agile projects

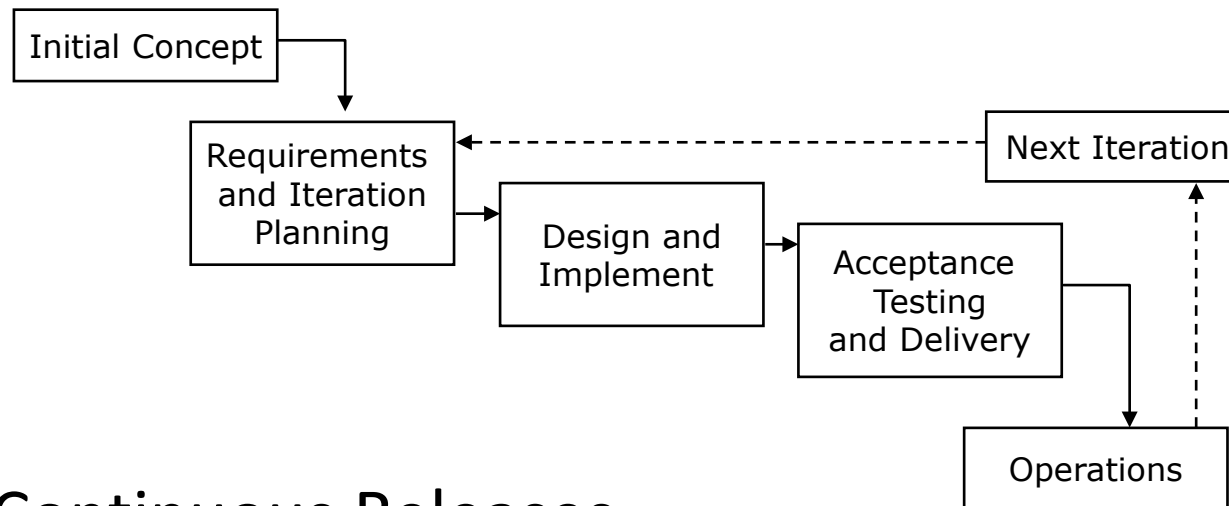


Agile Processes are Iterative

Agile Process Model



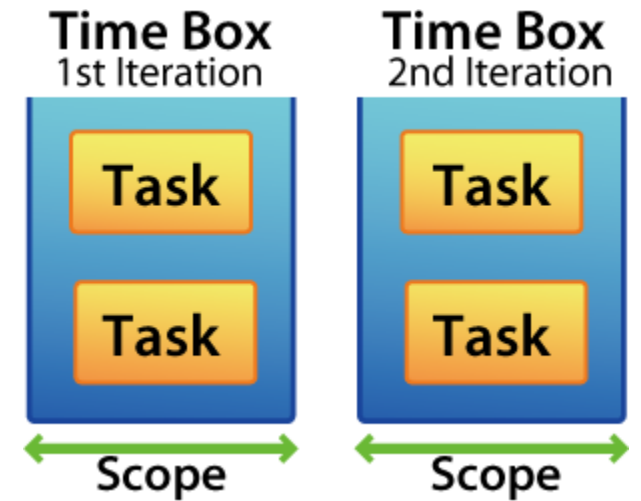
Iterative Waterfall Model



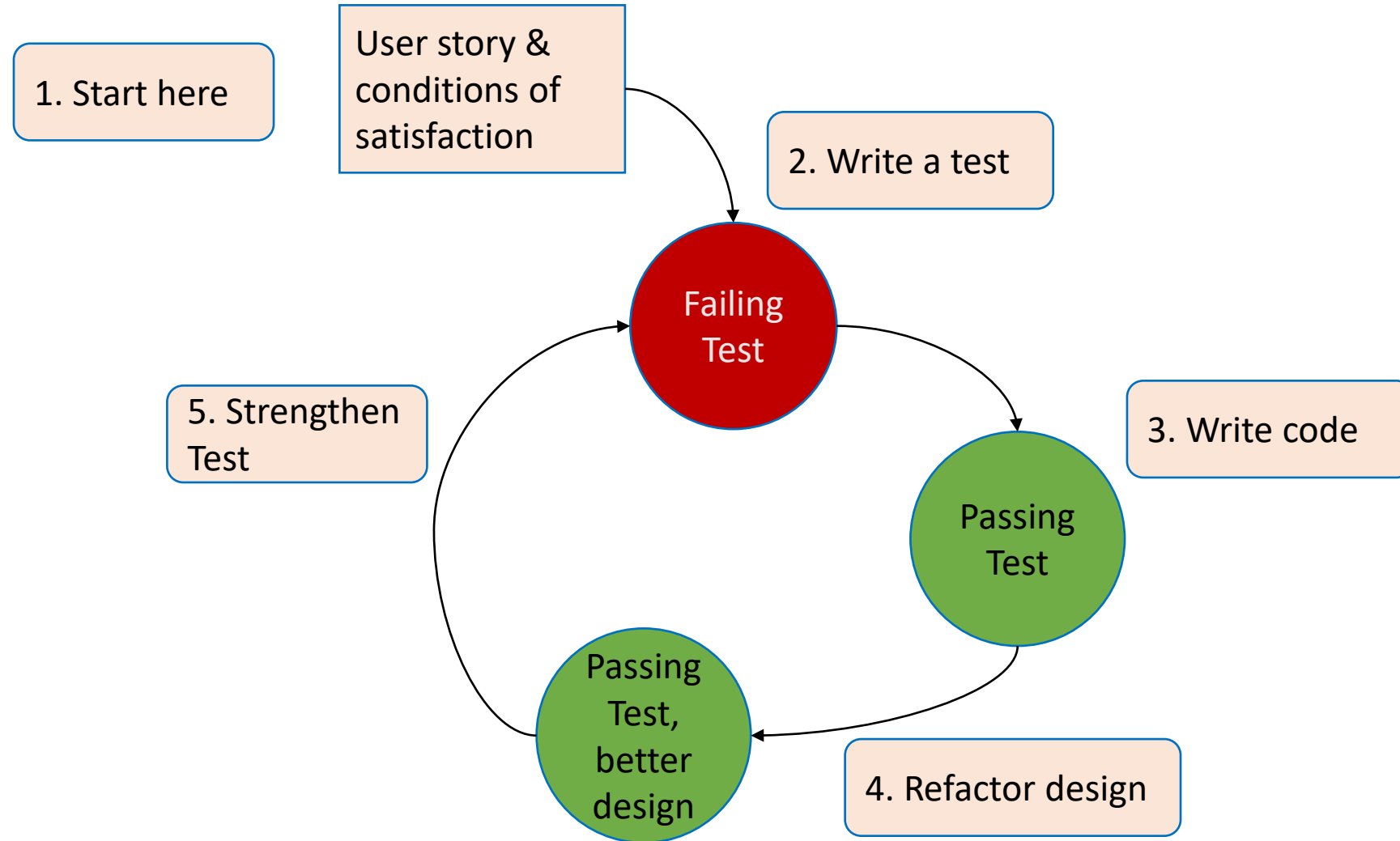
Key Idea: Small Continuous Releases

Agile Processes Reduce Risk by Time Boxing

- Each “iteration” is called a “sprint”
- Each sprint has a fixed duration
- Scope of features in a sprint is determined by the team
- Key insight: planning might be a guess at first, but gets better with time
- More on agile planning & estimation in the next module



Test Driven Development (TDD) is an Agile Practice

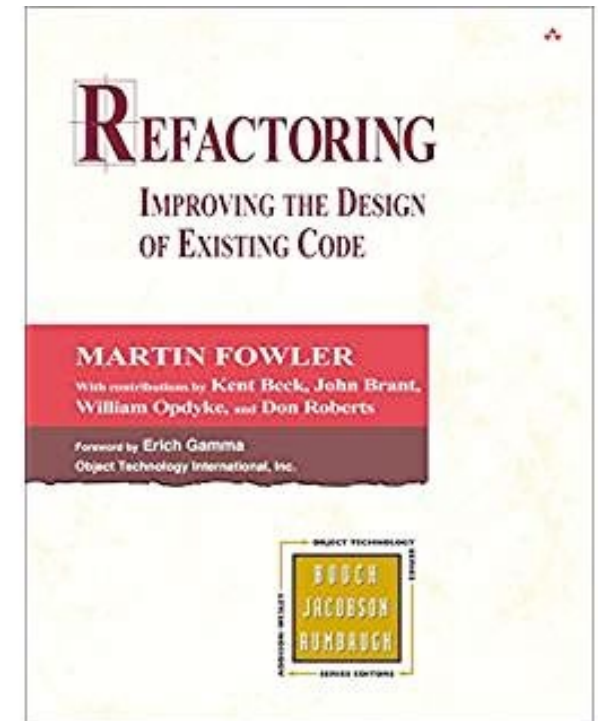


Code Refactoring is a gradual and continuous process that reduces technical debt

- Whenever you get a chance (while working on other features), if you see a **code smell**, fix it (i.e., take small steps)
- Use **IDE** to your advantage
- Rely on automated tests
- Separate refactoring from features
- For extensive refactoring: rely on **code reviews** where author explains the code/design to the team

The Book

- A catalogue of refactorings, similar to the design patterns in the GoF book
 - Names each transformation
 - Helpful for team communication
 - Names “bad smells” (triggers for refactorings)
 - Discusses when and how to apply refactorings and provides insight into tradeoffs



Be “agile” when designing your processes

- As an organization, you tweak your **process** as needed (from team sizes, to sprint duration, backlogs to change requests, stand up meetings to code review, and so on)

Company	Core Philosophy	Cycle Length	Key Structural Shift
Spotify	Autonomy & Alignment	Flexible (Team's choice)	Replaced managers with Squads, Tribes, and Guilds
Amazon	Radical Accountability	Continuous Delivery	"Two-Pizza Teams" aligned with microservices architecture
Basecamp	Deep Work / No Overhead	6 Weeks	Eliminated daily standups and standard backlogs
Enterprise Banks	Compliance-Driven Speed	2–4 Weeks	Hybrid Scrum with embedded regulatory experts

Agility and You

- In your project, you can display agility in some of the following ways:
 - Renegotiate specs
 - Reorder priorities
 - Alter implementation strategy
 - Improve team communication patterns
- If you are agile, you can adjust these things to deliver your product on time and get a good grade 😊

Learning Goals for this Lesson

- At the end of this lesson, you should be able to
 - Know the basic characteristics of the waterfall software process model
 - Be able to explain when the waterfall model is appropriate and when it is not
 - Understand how the waterfall and agile models manage risk
 - Be able to explain how agile process instill quality, including through test driven development